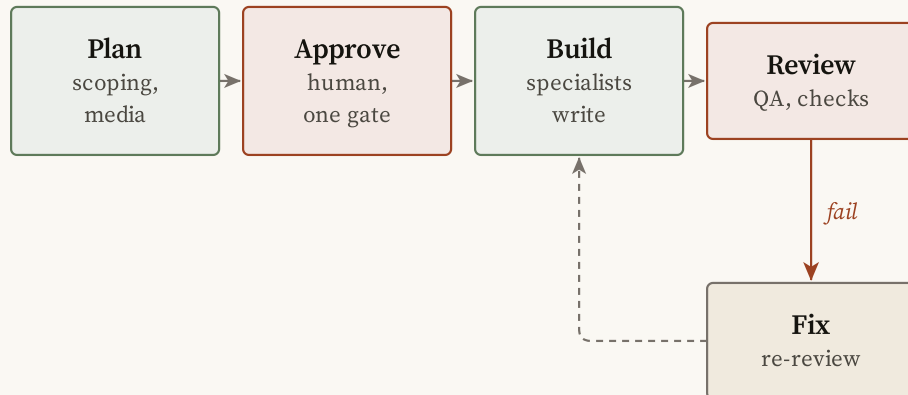


lesson-builder

A Claude Code skill that builds and updates interactive lessons under one quality bar and one human gate.

A lesson here is a small Vite and React app: LaTeX for notation, SVG for graphs, manim for animation, and a chat panel where a tutor agent answers questions about it. Hand-built, lessons like this drift apart. One gets careful diagrams and the next doesn't, an update to fix one section quietly breaks another that was already good, and there's no single point where a person actually looks before it ships. lesson-builder is six shared phases that a new lesson, an update, and a consolidation all run through, with specialists doing the writing and one mandatory gate where a person decides.

FIGURE 1 *the parts*



Review can send the build back for another pass; a second failure stops and asks a person.

Everything after the plan is agents writing in parallel to a scratch directory; the main agent only assembles and reviews.

The parts

PIECE	WHAT IT DOES
references/phase-0..5	Six shared phases: scoping, analysis, plan, build, review and fix, deploy.
content-orchestrator, medium-decider	Synthesize the research, then rank text versus diagram versus animation once, for the whole lesson.
graphics, manim, interactive-demo, web-image	Build one kind of media each, in parallel; interaction checks the interactive ones with Playwright.
code-review, content-review, visual-qa, scientific-accuracy	Check the assembly and each artifact independently, so no review softens because another passed.
_lesson-core/	The runtime shared by every lesson: chat prompt, response parser, sanitizer, proxy.

How one lesson is built

1. **Scope and analyze.** Phase 0 gets the lesson's shape from an interview or a `COURSE.md`. Phase 1 runs research workers in parallel, and the content-orchestrator synthesizes their findings into one analysis.
2. **Plan, then approve.** One medium-decider ranks text versus diagram versus animation for the whole lesson, once. The plan then goes to a person: the one gate the pipeline cannot skip.
3. **Build.** The specialists write in parallel into a scratch directory; the main agent assembles the result. An update splices the existing file with `scripts/lesson-ast.cjs` in a per-run git worktree, instead of rewriting it.
4. **Review, fix, deploy.** Code, content, visual and scientific review run on the assembly and each artifact; kept media from an update is re-checked too. A build-all pass and a headless Playwright check run before it commits and merges.

The hard parts

3.1 A tutor that can edit its own lesson

The chat panel's tutor agent can propose a change to the lesson it is teaching, not just answer questions about it, writing one of three tags into its reply that `processResponse.js` parses out of the model's text: `<<EDIT_GRAPH>>`, `<<SUGGEST>>` or `<<COMMIT_SUGGEST>>`. A suggestion goes to the learner for approval and is only committed once code-review reports zero blockers and zero majors against it. When a chat session forks into a side thread, the mutating tags are stripped first, and only a summary folds back, so a side conversation cannot commit anything on its own.

3.2 A review that means something, checked once

The eval for the content-review agent plants exactly one discourse defect into each of a set of lesson fragments, R01 through R12, hands the reviewer the fragment alone, and grades its output afterward against an answer key it never saw. A reviewer shown the defect it's meant to catch would just repeat it back; the eval only means something if it has to find that defect cold.

The same discipline shows up in what gets re-checked. `scripts/run-manifest.cjs` keys each pass or fail to the SHA-256 of the tree it ran against, so an artifact that already passed and hasn't changed is skipped by an attestation cache rather than re-reviewed, which is what makes it affordable to re-check kept media on every update instead of trusting it forever.

What it doesn't do

- The runtime tutor is its own orchestrator, not a subagent of the pipeline, because Claude Code's subagents cannot nest.
- The tutor runs off one prompt carrying every capability on every turn, rather than one scoped to what that turn needs; splitting it is planned, not done.
- The `<<TAG>>` protocol for tutor edits is a regular expression over model text, not a typed action channel.
- There is no multi-turn tutor eval, no prompt-injection fixture, and no learner eval that runs without someone asking for it.

From the repo <https://github.com/ihsan-sa/lesson-builder>: SKILL.md; references/phase-0-scoping.md through phase-5-deploy.md; agents/*.md; _lesson-core/ (processResponse.js, safeRender.js, server/proxy.js); scripts/lesson-ast.cjs; scripts/run-manifest.cjs; evals/teaching/ (README.md, expected-failures.json, R01-R12.md); ROADMAP.md, read 26 September 2026.